



Grado en Ingeniería Informática

Estructuras de Datos Avanzadas

Examen Convocatoria Extraordinaria 11 de junio de 2025

Nombre y apellidos	
DNI / Pasaporte	

Tiempo máximo: 2 horas

Instrucciones:

- La duración del examen es de 120 minutos.
- La puntuación de cada ejercicio se indica junto al enunciado.
- No se responderán preguntas sobre los enunciados. La interpretación de estos forma parte del examen.
- No está permitido el uso de calculadoras, ni la consulta de libros, apuntes o dispositivos electrónicos de ningún tipo durante el examen.
- Desde el Aula Virtual hay que descargar un archivo ZIP que contiene el esqueleto de los ejercicios del examen y las pruebas unitarias asociadas.
- El estudiante debe generar un fichero ZIP con el directorio en el que esté el código desarrollado durante el examen y entregarlo en la actividad "Examen práctico" disponible en Aula Virtual.

Consideraciones adicionales:

- No se pueden añadir nuevas clases, métodos o propiedades públicas.
- Solo se pueden añadir nuevas clases si son privadas. A las clases ya dadas solo se les puede añadir métodos y propiedades con visibilidad privada.
- No se pueden modificar las cabeceras de los métodos ya dados.
- El código del proyecto proporcionado se ha probado utilizando **Oracle OpenJDK 22.0.1** (disponible en el Escritorio de Desarrollo de MyApps) y **JUnit 5.8.1**.

Ejercicio 1 [2 puntos]

Implemente el iterador *ReverseInordenBTIterator*. Este iterador permite recorrer un árbol binario en *inorden* inverso. Es decir, primero el subárbol derecho; a continuación, la raíz; y, por último, el subárbol izquierdo. Dado el árbol binario de la Figura 1, el recorrido que realizaría un iterador de este tipo sería el siguiente: 6, 11, 2, 9, 3, 5, 10, 7. La clase se encuentra en el paquete *binaryTree*.

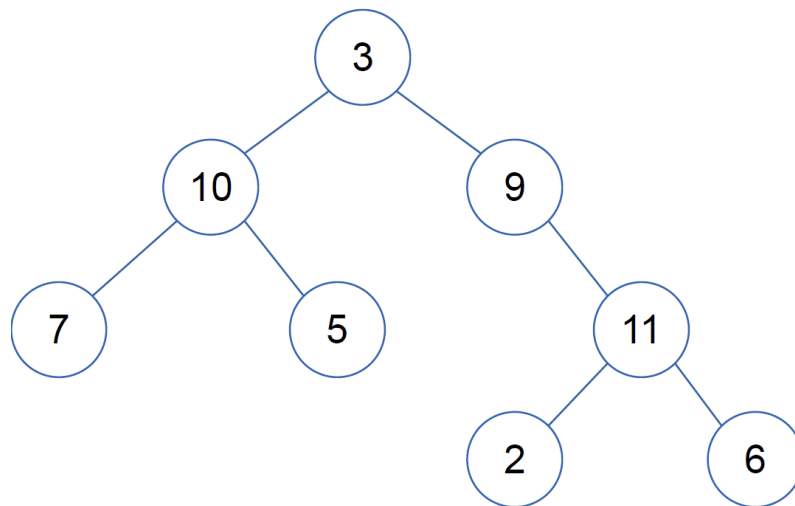


Figura 1. Ejemplo de un árbol binario.

Ejercicio 2 [2 puntos]

Implemente la función booleana *esPrefijo* que, dados dos árboles binarios, confirme si el primero es prefijo del segundo. Se dice que un árbol binario *a1* es prefijo de otro árbol binario *a2* cuando la parte inicial de *a2* equivale a *a1*. Es decir, si coinciden tanto el contenido como la forma (los mismos elementos en la misma disposición). En el ejemplo de la Figura 2, *a1* es prefijo de *a2*. La clase en la que se encuentra la función a implementar es *FunHandling*, que está en el paquete *binaryTree*.

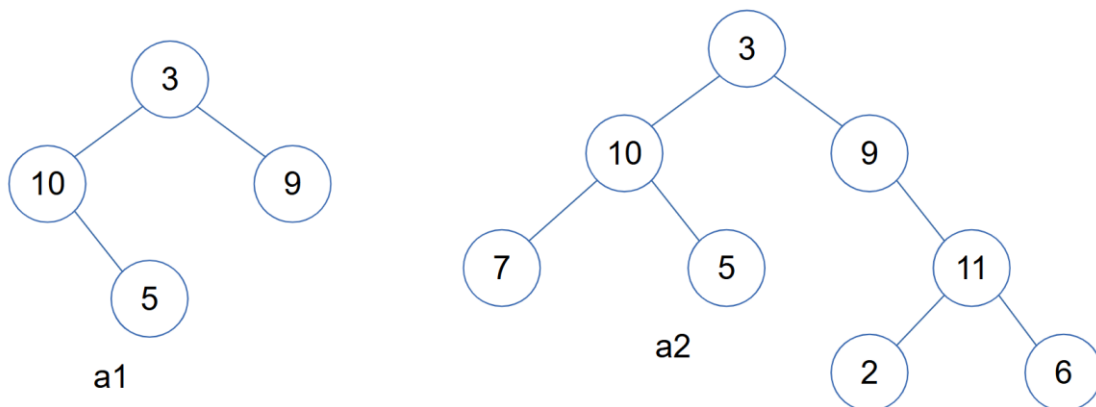


Figura 2. Ejemplo de dos árboles binarios, *a1* y *a2*. Nótese que *a1* es prefijo de *a2*.

Ejercicio 3 [6 puntos]

Red Eléctrica Española (REE) ha contratado a tu empresa para hacer un simulador de su sistema. Te han asignado el proyecto y has decidido emplear tus conocimientos de Estructuras de Datos Avanzadas para construir el simulador y evitar que nos volvamos a quedar a oscuras.

La red está organizada en áreas. Cada área es independiente, tiene un nombre único y contiene diferentes sistemas de generación de energía. Es decir, un área está compuesta por varias centrales eléctricas de diferente capacidad que pueden estar encendidas o apagadas. Dependiendo de cuántas centrales estén encendidas, el área estará generando más o menos energía. Cada central eléctrica genera una cantidad de energía diferente. De hecho, no hay dos centrales eléctricas dentro de la misma área que generen la misma potencia. Esto facilita la organización del área, que tiene ordenadas sus centrales eléctricas por la cantidad de energía que pueden generar.

Las diferentes áreas están conectadas entre sí mediante enlaces de alta tensión. REE puede establecer y deshacer estas conexiones entre las áreas.

Para la clase *Area*, se pide elegir la o las estructuras de datos más adecuadas para representar las áreas e iniciarlas debidamente en el constructor de la clase (0,25 puntos). Además, se pide implementar los siguientes métodos de la clase *Area*:

Método	Descripción	Puntos
public boolean encender (CentralElectrica c)	Recibe un objeto de la clase <i>CentralElectrica</i> . Si pertenece al área y no está encendida, intenta encenderla y devuelve <i>true</i> si lo consigue. En cualquier otro caso, devuelve <i>false</i> . Esta operación se realizará con complejidad $O(\log n)$.	0,30
public boolean apagar (CentralElectrica c)	Recibe un objeto de la clase <i>CentralElectrica</i> . Si pertenece al área y no está apagada, intenta apagarla y devuelve <i>true</i> si lo consigue. En cualquier otro caso, devuelve <i>false</i> . Esta operación se realizará con complejidad $O(\log n)$.	0,30
public boolean anadirCentral (CentralElectrica c)	Recibe un objeto de la clase <i>CentralElectrica</i> . Si no hay ninguna central en el área con la misma potencia, la añade al área y devuelve <i>true</i> . En cualquier otro caso, devuelve <i>false</i> . Esta operación se realizará con complejidad $O(\log n)$.	0,30
public SortedSet<CentralElectrica> encendidas ()	Devuelve una colección no modificable de tipo <i>SortedSet</i> de las centrales del área que están encendidas. Las centrales deben estar ordenadas de manera ascendente (de menor a mayor) por potencia.	0,25
public SortedSet<CentralElectrica> apagadas ()	Devuelve una colección no modificable de tipo <i>SortedSet</i> de las centrales del área que están apagadas. Las centrales deben estar	0,25

	ordenadas de manera ascendente (de menor a mayor) por potencia.	
public int generacion ()	Devuelve la cantidad de energía que el área está generando actualmente. Esta operación tiene que realizarse con una complejidad de $O(1)$.	0,05
public int potenciaMaxima ()	Devuelve la máxima cantidad de energía que el área podría producir si todas sus centrales estuviesen encendidas. Esta operación tiene que realizarse con una complejidad de $O(1)$.	0,05
public int completeShutDown ()	Apaga todas las centrales del área y devuelve el número de centrales que ha apagado. Esta operación debe tener una complejidad de $O(m)$, donde m es el número de centrales que estaban encendidas al llamar al método. No se pueden crear estructuras adicionales dentro del método para después recorrerlas. El consumo de memoria del método debe ser el mínimo imprescindible.	0,5
public int maxEnergy ()	Enciende todas las centrales del área y devuelve el número de centrales que ha encendido. Esta operación debe tener una complejidad de $O(m)$, donde m es el número de centrales que estaban apagadas al llamar al método. No se pueden crear estructuras adicionales dentro del método para después recorrerlas. El consumo de memoria del método debe ser el mínimo imprescindible.	0,5

Para la clase *RedElectrica*, se pide elegir la o las estructuras de datos más adecuadas para representar la red e iniciarlas debidamente en el constructor de la clase (0,25 puntos). Además, se pide implementar las siguientes operaciones en la clase *RedElectrica*:

Método	Descripción	Puntos
public void anadirArea (Area a)	Recibe un objeto de la clase <i>Area</i> y lo añade a la colección.	0,5
public boolean conectarAreas (Area a1, Area a2)	Recibe dos objetos de la clase <i>Area</i> . Conecta ambas áreas. Devuelve <i>true</i> si no estaban conectadas y las ha podido conectar. Devuelve <i>false</i> en cualquier otro caso.	0,5
public int	Devuelve la cantidad de energía que la red completa está generando actualmente.	0,5

energiaGeneradaActualmente ()		
public Collection<Area> areasActivas ()	Devuelve una colección con todas las áreas que están activas. Es decir, áreas que están produciendo energía.	0,5
public boolean colapsoDescontrolado (Collection<Area> c)	Recibe una colección de objetos de tipo <i>Area</i> y devuelve <i>true</i> si la red completa se apagaría en caso de que dichas áreas colapsaran . Esto sucede si se puede alcanzar el resto de las áreas de la red a partir de las que nos pasan en un único salto. Si un área colapsa , todas las áreas conectadas se apagan . Cuando un área se apaga, las áreas conectadas NO se apagan. Nótese la diferencia entre apagar y colapsar.	1

Para los métodos para los que se ha especificado una complejidad, es suficiente con alcanzar dicha complejidad. Por ejemplo, si se ha especificado que un método tiene que realizarse con complejidad $O(\log n)$, no es necesario hacerlo con complejidad $O(1)$. En aquellos métodos en los que no se ha especificado la complejidad, se valorará conseguir la menor complejidad posible.